

实验报告一

一、问题回答部分

PART1:

- 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？
- 答：有哦、，运用 WPS 共同编辑报名文档。
- 思考一下，Git 为什么要设计“暂存-提交”两个步骤？
- 答：暂存：精确挑选、组合、排列即将入库的内容；提交：将排好版的内容固化为不可变的历史记录。
- `git branch` 和 `git branch -a` 的区别是什么？查阅资料并回答。
- 答：`git branch` 与 `git branch -a` 的核心区别在于查看范围：前者仅列出本地分支，后者则同时显示本地分支和远程跟踪分支（通常带有 `remotes/origin/` 前缀）。在日常使用中，`git branch` 适用于管理自己创建的本地分支，而 `git branch -a` 更适合排查远程分支状态或确认 `fetch` 是否完整；若只需查看远程分支可使用 `git branch -r`。

PART2:

我选择：

- [Commit Message 规范](#)
- [Git Flow 分支控制](#)

1.两个网页内容的简单概括

关于“怎么写提交说明”

这个网页讲的是，每次保存代码修改时写的备注不能太随便。不能只写“改了代码”或者“修了问题”这种模糊的话，而要清楚地说明“具体改了什么”以及“为什么要改”。这样以后自己或别人回头看的时候，一眼就能明白当时的修改目的，也方便在出问题时快速找到对应的改动，还能让团队里的其他人更容易看懂你的工作。

关于“怎么管理不同的开发任务”

这个网页介绍了一套多人一起写代码时的分工方法。就像做饭有备菜区、烹饪区和出餐区一样，它把代码分成了不同的“工作区”：有的专门用来做新功能，有的用来测试，有的只放稳定能用的版本，还有的用来紧急修补线上问题。这样做能让每个人各司其职，互不打扰，避免大家同时改代码时互相覆盖或搞乱已经上线的内容，让整个开发过程更有条理。

2.为什么要学习 Git

结合上面两点，我觉得学 Git 不只是学一个存代码的工具，更是学一种和别人一起把事情做好的方法。

首先，它能帮我们养成认真记录的习惯。就像写日记要写清楚时间、事件和感受一样，规范的提交说明让每一次代码修改都有迹可循，不再是只有自己看得懂的“天书”。这既是对自己的代码负责，也是对后续接手的人尊重。

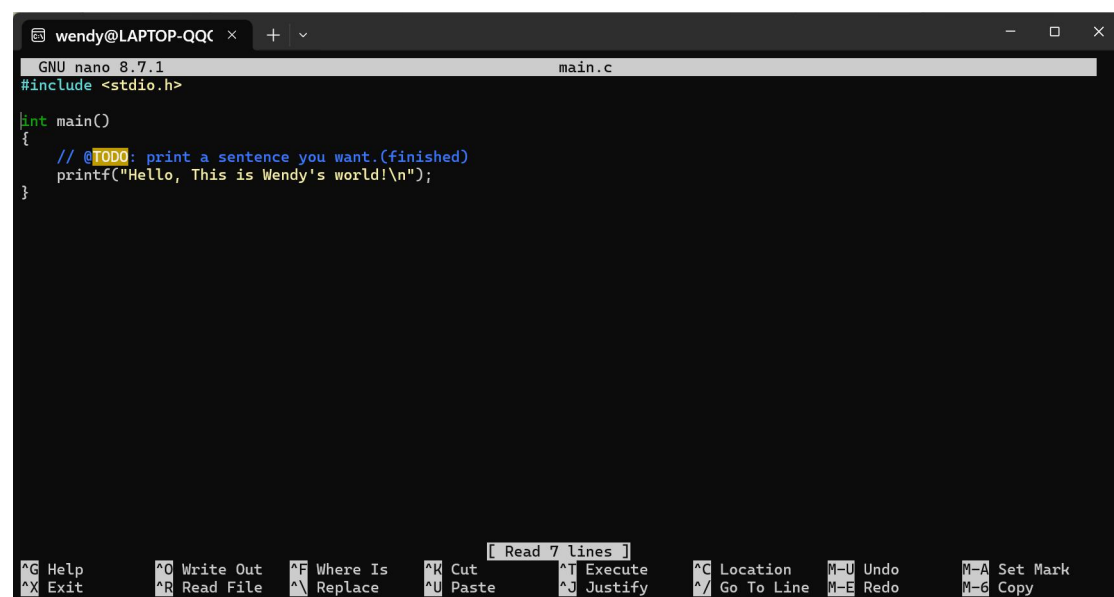
其次，它能让团队合作变得顺畅。当几个人甚至几十个人一起做一个项目时，如果没有统一的规则，很容易各干各的、互相冲突。Git 提供的这套分工方法，就像给团队定好了交通规则，让大家知道什么时候该做什么、代码该放在哪里，减少了混乱和返工。

最后，它其实是在教我们如何靠谱地完成一件事。无论是写清楚每一次改动的原因，还是按流程管理不同阶段的代码，本质上都是为了让整个过程更可控、结果更可靠。学会用 Git，其实是学会了用一种有条理、可追溯、能协作的方式去解决问题——这种思维方式，不管以后是否继续写代码，都会让我们受益。

二、实验步骤（&必要截图）

1. 配置环境、注册个人 github 账户，连接 VScode 与 wsl
2. 编辑 sentence 时 VScode 出现了受限问题，于是转换到 Ubuntu 窗口完成

然后发现 git 还没有克隆于是先克隆后再进行了修改和编译



```
wendy@LAPTOP-QQC x + v
GNU nano 8.7.1 main.c
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.(finished)
    printf("Hello, This is Wendy's world!\n");
}
```

3. 创建新分支 feature 并进行了对 printf 的修改→“Have a nice holiday!”
4. 当我再次打开 main.c 时候发现里面内容不变这说明了分支之间的独立性
5. 再一次我修改了 main.c 中的内容为“Hello This is me”
6. 当进行 merge 操作时出现了冲突

```
wendy@LAPTOP-QQC x + v
GNU nano 8.7.1 main.c
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.(finished)
    <<<<<< HEAD
    printf("Hello, This is me!\n");
    =====
    printf("Have a nice holiday!\n");
    >>>>>> feature
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^M Set Mark
^X Exit      ^R Read File  ^N Replace    ^J Paste      ^_ Justify    ^/ Go To Line ^E Redo       ^G Copy
```

7. 出现冲突的原因：修改了同一行 printf，GIT 无法自动判断取舍于是通过 conflict 提醒开发者手动处理

```
wendy@LAPTOP-QQ0JCPG5:~/Wendy-s-SCI-work$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
wendy@LAPTOP-QQ0JCPG5:~/Wendy-s-SCI-work$ |
```

8. 解决冲突完成合并提交

```
wendy@LAPTOP-QQC x + v
GNU nano 8.7.1 main.c *
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.(finished)
    printf("Hello, This is me!Have a nice holiday\n");
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^M Set Mark
^X Exit      ^R Read File  ^N Replace    ^J Paste      ^_ Justify    ^/ Go To Line ^E Redo       ^G Copy

wendy@LAPTOP-QQ0JCPG5:~/Wendy-s-SCI-work$ git add main.c
wendy@LAPTOP-QQ0JCPG5:~/Wendy-s-SCI-work$ git commit -m"merge feature and resolve merge conflict"
[main 96459b9] merge feature and resolve merge conflict
wendy@LAPTOP-QQ0JCPG5:~/Wendy-s-SCI-work$ |
```